What Happens When You Turn a Card Game into an NP-Complete Problem? (A Personal Exploration)

Overview

From an engineering perspective, I designed and implemented a turn-based PvE card game using the Unity engine. The game features simple rules yet requires both luck and strategic decision-making. The computer opponent's strategy is powered by a locally deployed server running a trained ensemble AI model.

Theoretically, I abstracted the game mechanics into a subsequence selection and score maximization problem governed by specific rules. Through in-depth analysis, I discovered that the actual decision space grows combinatorially with the number of moves, leading to an explosive increase in complexity. To understand its computational complexity, I examined a generalized version of this card game. Suspecting the abstracted problem might be NP-Hard, I formally reduced it to the Subset Sum problem, thereby proving that the problem is not only NP-Hard but also NP-Complete. Consequently, exact solutions become computationally infeasible even for moderately sized instances.

To address this challenge, I explored a general-purpose solution approach based on a lightweight surrogate model that integrates a Transformer architecture with a genetic algorithm. The core idea is to leverage an exact exhaustive-search algorithm to generate high-quality training samples within a computationally tractable problem scale, then train a computationally inexpensive AI model to mimic the optimal decision-making behavior observed in those samples.

This document is not a formal academic paper, nor does it claim to present rigorously validated academic contributions. Rather, it functions as a high-fidelity project report or a personal portfolio piece—demonstrating both theoretical understanding and engineering capabilities within a carefully designed sandbox environment. The work achieves a meaningful depth of exploration and incorporates several innovative elements, aiming to showcase my proficiency and readiness in applying advanced computational techniques to complex real-world-inspired problems.

Project Highlights

- **Abstraction and Implementation of Problem Modeling**
- **Rigorous Algorithmic Analysis**
- **Proof of NP-Completeness for a Custom-Designed Problem, with Supporting Discussion**
- **Comprehensive AI Model Evaluation Framework**
- **Design, Implementation, and Analysis of a Surrogate Model, including Discussions on Generalization and Robustness**
- **Batch Processing and Multithreading Optimizations for the Ensemble Model**
- **Custom Development and Extension of a Relatively Complex Game Framework**
- **End-to-End System Design and Implementation**

- **Discussion of a Tree-Based Representation Well-Suited for Expressing Combinatorial Problems (Appendix)**

> Note: The specific implementation of the code is available in the source code of this project. The dataset itself is not uploaded, but the code used to generate the dataset is included.

Document Structure

This document is divided into four main sections:

1. Game Rules and Algorithmic Analysis

This section provides a detailed description of the game rules and their programmatic implementation, including:

- **Game Rules**
- **Algorithmic Analysis**
- **Proof of NP-Completeness**
- **Discussion on Algorithmic Optimizations and the Proposal of an AI Surrogate Model**

2. AI Technical Details

This section delves into the inner workings of the AI system used in the game and its core implementation, covering:

- **AI Model Implementation**
- **Model Training**
- **Ablation Studies on Data Preprocessing**
- **Model Evaluation**
- **Ensemble Model Evaluation and Optimization**
- **Discussion on Modeling Paradigm Conflicts and Inductive Biases in the Transformer Architecture**

3. Server Deployment and Game Architecture

This section presents the Unity client architecture and live gameplay demonstration, including:

- **AI Model Server Setup**
- **Client-Side Game Architecture**
- **Live Gameplay Demonstration**

4. Appendix, Author’s Self-Reflection (Q&A), and Other Supplementary Materials

Part I: Game Rules and Algorithmic Analysis

Game Rules and Gameplay Example

The game rules are somewhat similar to "Tile Matching" (card game). Players score points by pairing cards to eliminate them. When it's the computer's turn, the current game state is captured, and the dynamic card layout data is sent to a local AI server. The AI returns its predicted optimal move, which is then automatically executed and displayed to the user on the client side. After three rounds, the player with the higher total score wins.

Initial Setup

1. The system creates a list **P** containing 52 numbers, where each integer from 1 to 13 appears exactly four times (13 × 4 = 52 elements in total).
2. **Six distinct elements** are randomly drawn from list **P** to form list **A**.
    - **Example:** Suppose we obtain **A = {1, 2, 3, 7, 4, 6}**.
    - **(Note)** In this specific study, the number of elements in **A** is fixed at 6.
3. Then, **three elements (allowing duplicates)** are randomly drawn from the remaining elements of **P** to form list **B**.
    - **Example:** Suppose we obtain **B = {4, 3, 9}**.
    - **(Note)** In this specific study, the number of elements in **B** is fixed at 3.

Insertion Positions

- Valid insertion positions include all gaps between elements of **A** as well as both ends.
- For the example **A = {1, 2, 3, 7, 4, 6}**, the insertion positions (numbered) are: ① 1 ② 2 ③ 3 ④ 7 ⑤ 4 ⑥ 6 ⑦
- There are a total of 6 + 1 = 7 possible insertion positions.

Start of a Turn

- On their turn, a player selects one card from **B** and inserts it into one of the valid insertion positions in **A**.
- Once selected and inserted, the card is removed from **B** and added to **A**.

Example Turn Sequence

First Move

1. The player chooses to take the card 3 from **B** and insert it into position ① of **A** (i.e., before **A[0]**).
2. **A** and **B** are updated:
    - **A** becomes **{3, 1, 2, 3, 7, 4, 6}**
    - **B** becomes **{4, 9}**

Matching Rule

- After insertion, examine the newly inserted card (3). Search **A** for any other element with the same numerical value.
- **Example:** The newly inserted card is 3. In **{3, 1, 2, 3, 7, 4, 6}**, **A[3]** also equals 3, forming a match with the newly inserted 3.
- **(Note)** At most one matching element exists in **A** for any newly inserted card. This is guaranteed because **A** initially contains only unique values.

Scoring and Removal (First Move)

1. Upon finding a match, define a sequence starting from the newly inserted card up to (and including) the matched element.
    - **Example:** The new 3 is at **A[0]**, and the matched 3 is at **A[3]**. The sequence is **A[0]** through **A[3]**: **{3, 1, 2, 3}**.
2. This matched sequence is removed from **A**.
3. Score is calculated as the sum of all values in the sequence.

    - **Example:** $Score_1 = 3 + 1 + 2 + 3 = 10$.
4. **A** is dynamically updated:
    - After removing **{3, 1, 2, 3}**, **A** becomes **{7, 4, 6}**.
    - **B** remains unchanged as **{4, 9}**.
5. Insertion positions are updated accordingly: for **A = {7, 4, 6}**, valid positions become ① 7 ② 4 ③ 6 ④.

Second Move

1. The player chooses to take the card 9 from **B** and insert it into position ④ of **A**.
2. **A** and **B** are updated:
    - **A** becomes **{7, 4, 6, 9}**.
    - **B** becomes **{4}**.
3. Check for matches: the newly inserted card is 9. In **{7, 4, 6, 9}**, no other element equals 9.
4. No match occurs, so $Score_2 = 0$.
5. **A** and **B** remain in their current states.
6. Insertion positions become ① 7 ② 4 ③ 6 ④ 9 ⑤.

Third Move

1. The player takes the last remaining card 4 from **B** and inserts it into position ⑤ of **A**.
2. **A** and **B** are updated:
    - **A** becomes **{7, 4, 6, 9, 4}**.
    - **B** becomes **{}** (empty).
3. Check for matches: the newly inserted card is 4. In **{7, 4, 6, 9, 4}**, **A[1]** also equals 4, matching the newly inserted **A[4]**.
4. The matched sequence is from **A[1]** to **A[4]**: **{4, 6, 9, 4}**.
5. Score is recorded: $Score_3 = 4 + 6 + 9 + 4 = 23$.
6. After removing **{4, 6, 9, 4}**, **A** becomes **{7}**.
7. **B** is already empty (**{}**).

End of Turn

- All players (in this example, just the human player) have played three cards, and all elements from **B** have been inserted into **A**.
- The turn ends.

Total Score

- The total score is the sum of all scores obtained during the turn.
- **Example:** Total Score = $Score_1 + Score_2 + Score_3 = 10 + 0 + 23 = 33$.

Exhaustive Recursive Algorithm Analysis

To compute the optimal strategy for a given pair of lists **A** and **B**, we first clarify the following key concepts:

Given A and B:

- **A** is an integer list of length m, with all elements distinct, each in the range 1 to K.
- **B** is an integer list of length n, with elements allowed to repeat, each also in the range 1 to K.

Strategy:
In this context, a *strategy* specifies **both**:

1. The **order** in which elements from **B** are inserted, and
2. The **insertion positions** in **A** where each selected element is placed.

Thus, a complete strategy fully determines the sequence of moves: for each element taken from **B**, we decide *when* (in which turn) to play it and *where* (at which valid position) to insert it into the current **A**.

Matching:
Immediately after selecting a card from **B** and inserting it into a specific position in **A**, we perform the following check:
① Whether the value of the newly inserted card matches the value of **any pre-existing card** already present in **A**.
② If such a matching card exists in **A**, we say a **"match"** has occurred.

Match Sequence and ***Removal***:
When a match occurs, a **"match sequence"** is identified. This sequence includes:
① The newly inserted card,
② The pre-existing card in **A** with the same value, and
③ All cards located **between** these two cards in the current list **A**.

This entire contiguous subsequence is then **removed** from **A**.

Score:
The score for this move equals the **sum of all values** in the removed match sequence.

No Match and Zero Score:
If the value of the newly inserted card does **not** appear anywhere else in the current **A**, then **no match** occurs. In this case, the match sequence is empty, nothing is removed from **A**, and the score for this move is **0**.

Total Score:
Applying a given strategy means sequentially inserting all elements of **B** into **A** according to the specified order and positions. Each insertion triggers a match check, yields a score (possibly zero), and potentially modifies **A**. The **total score** is the sum of the scores from all *n* insertions (in our specific problem, *n* = 3).

Best Strategy:
The **best strategy** is the one that **maximizes the total score** over all possible strategies.

First, we perform a generalized mathematical analysis of the problem:

Strategy Evaluation Complexity:

This computes the time complexity required to determine the outcome—according to the game rules—given arrays A and B and a specified insertion position(strategy).

For each element in **B**, the algorithm attempts to insert it into every possible position in the current **A**, then recursively processes the remaining elements. Suppose $|B| = n$ and $|A| = m$. In the first step, there are n choices for which element to insert, and for each such choice, there are $m + 1$ possible insertion positions. Thus, there are $n \cdot (m + 1)$ possible paths at the first level.

After inserting an element at an arbitrary position, let p denote the probability that a match occurs.

Expected Length of the Removed Subsequence Upon a Match

We now analyze the expected length of the subsequence removed when a match occurs.

Consider the following simplified scenario:
We have a sequence **A** of m distinct integers. A value $x \in A$ already exists in **A**. Now we insert another copy of x into a uniformly random position among the $m+1$ possible insertion slots, forming a new sequence **A'**. This creates a contiguous subsequence $[x \dots x]$, which is then removed. What is the expected length of this removed subsequence?

This can be further reduced to a purely combinatorial problem:

> From the integers $1, 2, \dots, m+1$, select two distinct numbers uniformly at random. What is the expected number of integers between them, inclusive?

Let the two selected positions be i and j with $i < j$. The distance (inclusive length) is $d = j - i + 1$, where $2 \le d \le m+1$.

The expected value is:

$$\mathbb{E}[d] = \frac{\sum_{d=2}^{m+1} d \cdot (m-d+2)}{\binom{m+1}{2}} = \frac{(m+2)\sum_{d=2}^{m+1} d - \sum_{d=2}^{m+1} d^2}{\binom{m+1}{2}}$$

Applying the formulas for the sum of an arithmetic series and the sum of squares, and simplifying, we obtain:

$$\mathbb{E}[d] = \frac{m+5}{3}$$

Therefore, **the expected length of the removed subsequence upon a successful match is** $\frac{|A|+5}{3}$.

Consequently, if the current length of **A** is $|A|$, then after a match, the expected new length $|A'|$ is:

$$\mathbb{E}[|A'|] = |A| + 1 - \frac{|A|+5}{3} = \frac{2|A|-2}{3}$$

Expected Matching Probability Over Time

Now consider the matching probability dynamics.

We have a sequence **A** of length m with all distinct elements. We insert an element b (from **B**, which may contain duplicates) into a random position in **A**. With probability p, a match occurs; otherwise (with probability $1-p$), nothing is removed.

Let:

- $E[K]$: expected number of elements in **A** that can match any element in the remaining **B**.
- $E[K']$: expected number of such matchable elements in **A** after a successful match.

Due to the linearity of expectation and the earlier result on sequence length reduction, after a match:

$E[K'] \approx \frac{2E[K]-2}{3}$

Meanwhile, the size of **B** decreases by 1.

The matching probability at any step is approximately:

$p = \frac{\text{number of matchable elements in } A}{\text{size of remaining } B}$

Thus, after a successful match:

- Numerator (matchable count) shrinks roughly by a factor of $2/3$,
- Denominator (size of **B**) decreases by 1.

Assuming large values ($x = E[K] \gg 1$, $y = |B| \gg 1$), we observe:

$\frac{x}{y} > \frac{(2x-2)/3}{y-1} \approx \frac{2x}{3(y-1)} \approx \frac{2}{3} \cdot \frac{x}{y}$

Hence, **the matching probability p decreases after a successful match**.

Conversely, if no match occurs:

- **A** grows by 1 (since the new element is added without removal),
- The new element may itself become a future match target, so the number of matchable elements **increases or stays the same**,
- **B** still shrinks by 1.

Therefore, **p increases significantly after a failed match**.

This leads to a key insight:

> The system exhibits a **negative feedback loop**:
>
> - If a match occurs → p decreases;
> - If no match occurs → p increases.

This behavior aligns with intuition and stabilizes the system.

State Evolution Model

Define M_t as the length of **A** at the beginning of step t. The system evolves as:

- With probability p_t (match succeeds): $M_{t+1} = \frac{2M_t-2}{3}$
- With probability $1 - p_t$ (no match): $M_{t+1} = M_t + 1$

Here, p_t depends on the overlap between the current **A** and the remaining **B**.

> Due to the negative feedback described above, the system self-regulates:
>
> - Elimination reduces M → lowers future p_t;
> - No elimination increases M → raises future p_t.

Thus, M_t neither explodes nor collapses to zero, but fluctuates around a **dynamic equilibrium**.

Steady-State Analysis

Assume the system reaches a steady state where the expected length remains constant: $\mathbb{E}[M_t] \approx M$ for all t. Approximate $p_t \approx p$ as a constant depending only on M and the current size of **B** (denoted y).

Then:

$$M = p \cdot \frac{2M-2}{3} + (1-p)(M+1)$$

Solving for M:

$$M = \frac{3}{p} - 5$$

Estimating the Equilibrium Length

To estimate p and locate the balance point, we adopt a **mean-field approximation**:

- Although **B** is a fixed input sequence, **A** evolves dynamically.
- Both share the same alphabet (values 1 to K).
- In expectation, the number of matchable elements between **A** and the remaining **B** is proportional to M.

We make two key assumptions:

1. **Mean-field approximation**: In the large-system limit, the instantaneous matching probability can be approximated by its expected value, and this probability is proportional to the current length M of sequence A.
2. **Quasi-static approximation**: Since $y = |B|$ changes slowly (decreases by 1 per step), the system quickly equilibrates at each fixed y.

Thus:

$$p_t \approx c \cdot \frac{M_t}{y} \quad (\text{for some constant } c)$$

In equilibrium, $M \approx 3/p - 5 \approx 3/p$ (ignoring the constant for scaling analysis). Substituting $p \approx M/y$:

$$M \approx \frac{3}{p} \approx \frac{3y}{M} \quad \Rightarrow \quad M^2 \approx 3y$$

Therefore, for most of the process (when $y = \Theta(n)$):

$$M_t = \Theta(\sqrt{n})$$

This implies that the typical length of the dynamic sequence **A** scales with the square root of the initial size of **B**.

Since $M_t = \Theta(\sqrt{n})$ holds in typical cases, the total time complexity is:
$T = \sum_{t=1}^{n} O(M_t) \approx \sum_{t=1}^{n} O(\sqrt{n}) = O(n \cdot \sqrt{n}) = O(n^{3/2}).$

More precisely, if we treat $M_t \approx \sqrt{n-t}$ as a continuous function, then:
$T \approx \int_0^n \sqrt{t}, dt = \frac{2}{3} n^{3/2} = \Theta(n^{3/2}).$

This model is primarily intended for analyzing the macroscopic statistical behavior of the system when $M \gg 1$. We assume that the probability of the system being completely emptied [i.e., $M = 0$ or $M = 1$] during execution is negligible. The validity of this assumption stems from the fact that the equilibrium point established by the negative feedback mechanism, $M_{\text{eq}} \approx \sqrt{3y/c}$, decreases slowly as y diminishes, yet remains far from zero as long as y is still relatively large. Consequently, the system spends the vast majority of its time in states where M is sufficiently large.

Our analysis characterizes the system's behavior in this steady-state regime and does not precisely capture the dynamics near absorbing or reflecting boundaries at very small values of M—for example, if all elements of A are completely eliminated after being matched by a subset of B, then subsequent insertions of B would cause elements from B to match among themselves.

Research on a Specific Instance of the Original Problem

In order to obtain concrete data for training AI and implementing a Unity game, we investigate a specific instance of the original problem where array A has length 6, array B has length 3, and K = 13.

Problem Modeling

First, we develop the `simulate_insertion_tool()` function capable of handling the insertion process and scoring. Since array A dynamically changes due to insertions and subsequent matches, when this function is called, the input length of A is no longer the initial length of 6, but rather ranges from 0 to 8 (as A can only be in the range 0–8 when `insertion` might be called—A cannot be negative, and when A reaches length 9, B becomes empty and no further elements can be inserted from B). After insertion and any resulting matches, A's length will be in the range 0–9.

We define the function `simulate_insertion_tool(A, b, pos)` (The implementation in source code), which inserts element `b` into list `A` at the specified position `pos` and handles any matching that may occur.

The function returns: `[score, length_removed_due_to_matching, new_length_of_A, match_occurred (0 or 1), new_A]`.

One key challenge in this problem is designing an appropriate data structure to represent insertion strategies.

The strategy structure I use is a 3×2 two-dimensional matrix (or list), which we refer to as `move`. Each row of this matrix represents one insertion operation, with two columns:

- The first column indicates the index of the element in array B;
- The second column specifies the position in the current array A where this element should be inserted.

Importantly, the "insertion position" is always relative to the most up-to-date state of A, not the initial A.

For example:

```
B = [1, 2, 3]
move = [[1, 6], [0, 3], [2, 0]]
```

This strategy means:

- Step 1: Insert B[1] (i.e., element 2) into the current A at index position 6;
- Step 2: Insert B[0] (i.e., element 1) into the updated A at index position 3;
- Step 3: Insert B[2] (i.e., element 3) into the A (updated again) at index position 0.

Each step operates on the current version of A, and every insertion of B[i] causes A to be updated due to both the insertion itself and any subsequent matching.

Thus, we can design a recursive algorithm:

`Recursive_StrategyAndScore(A, B, original_indices=None, move_sequence=None, used_indices=None)` (implementation available at source code)

which returns: **the theoretical maximum score** and **the corresponding sequence of moves that achieves this maximum score**.

Estimating the Scale of Exhaustive Search for Optimal Decision Paths under the Specific Instance

We compute the total number of possible final static configurations when inserting all elements of B into A, assuming no specific values for A and B are given and **no matching or elimination occurs at all**. Note that this count represents only the number of distinct *final static arrangements*, not the full decision process. Through static modeling, we estimate the theoretical upper bound on the number of possible final hand combinations.

Consider the scenario where we insert all elements of B (an array of length 3) into A (an array of length 6), ignoring the actual numeric values (i.e., treating cards only as belonging to group A or group B). This becomes a multiset permutation problem: how many distinct length-9 arrays can be formed by arranging the multiset {A, A, A, A, A, A, B, B, B}?

The answer is: $\binom{9}{6} \cdot \binom{3}{3} = \binom{9}{6} = \binom{9}{3}$

Now, we assign actual numeric values (from 1 to 13) to these placeholders:

- Each of the 6 A-cards must have **distinct** values from 1 to 13, so the number of ways to assign values to A is the number of permutations: $P(13, 6) = \frac{13!}{(13-6)!}$.
- Each of the 3 B-cards can take **any** value from 1 to 13, with repetitions allowed, giving 13^3 possible assignments.

Therefore, the total number of possible final configurations (under the no-matching assumption) is:
$\binom{9}{3} \cdot \frac{13!}{(13-6)!} \cdot 13^3 = 228{,}012{,}744{,}960$

Assuming each sample requires approximately 10 bytes of storage, the total data size would exceed **2,000 GB**—an enormous amount.

> **Important Caveats**:
> This is only a rough estimate. As noted earlier, in the actual game, each insertion may trigger matching and removal, dynamically altering the length and content of A. Consequently, the state of A changes after each insertion, meaning that the order of insertions matters significantly.
>
> For example, inserting b_1 at position 1 followed by b_2 at position 2 can yield a different outcome than inserting b_2 at position 2 first and then b_1 at position 1—due to intermediate matching effects. Thus,

> these are two distinct decision paths.
>
> Moreover, our calculation assumes **no matches occur**, which also implies that some insertion sequences might be invalid in practice (e.g., attempting to insert at a position beyond the current length of A). Matching not only changes A's length but also introduces complex state dependencies. It remains unclear whether matching ultimately increases or decreases the total number of feasible decision paths—but it certainly makes the true search space highly non-trivial and far more intricate than this static upper bound suggests.

A Better Estimate of Data Scale via Monte Carlo Simulation

To estimate the real impact of matching, we employed probabilistic statistical analysis—specifically, Monte Carlo simulation—for the specific case where `length(A) = 6` and `length(B) = 3`.

In each trial:

- Random arrays A (length 6) and B (length 3) are generated.
- One card is randomly selected from B (without replacement) and inserted at a random valid position in A.
- The `simulate_insertion_tool` function is called to perform the insertion and handle any resulting matches.
- This process is repeated **twice**: the second insertion operates on the updated A and reduced B from the first step.
- We record the third return value of `simulate_insertion_tool`—the new length of A after matching.

This entire procedure was repeated **20 million times** (`total_times = 20,000,000`) for statistical robustness. Below is the Python-like pseudocode used:

```
{
    A, B = get_A_and_B_random()

    card = random.choice(B)
    pos = random.randint(0, 6)
    _, _, new_length1, _, newA = simulate_insertion_tool(A, card, pos)
    all_lengths1.append(new_length1)

    B.remove(card)
    card = random.choice(B)
    pos = random.randint(0, len(newA))
    _, _, new_length2, _, newA = simulate_insertion_tool(newA, card, pos)
    all_lengths2.append(new_length2)
}
```

Simulation Results (20M trials):

Distribution of A's length after the first insertion (`A1`):

- `length(A1) = 0`: 559,058 times (2.80%)
- `length(A1) = 1`: 1,112,357 times (5.56%)
- `length(A1) = 2`: 1,669,068 times (8.35%)
- `length(A1) = 3`: 2,216,588 times (11.08%)
- `length(A1) = 4`: 2,775,405 times (13.88%)
- `length(A1) = 5`: 3,333,920 times (16.67%)
- `length(A1) = 7`: 8,333,604 times (41.67%)

Expected value of `length(A1)`: 4.8603

Distribution of A's length after the second insertion (`A2`):

- `length(A2) = 0`: 689,171 times (3.45%)
- `length(A2) = 1`: 1,746,955 times (8.73%)
- `length(A2) = 2`: 2,511,990 times (12.56%)
- `length(A2) = 3`: 3,001,901 times (15.01%)
- `length(A2) = 4`: 3,202,618 times (16.01%)
- `length(A2) = 5`: 3,129,061 times (15.65%)
- `length(A2) = 6`: 3,434,722 times (17.17%)
- `length(A2) = 8`: 2,283,582 times (11.42%)

Expected value of `length(A2)`: 4.1555

> **Note**: We only perform **two** `simulate_insertion_tool` calls per trial because:
>
> - After the first insertion, B has 2 elements left.
> - After the second insertion, B has 1 element left.
> - The third (final) insertion would occur next, but since our focus is on estimating the *branching factor* of the decision tree, and because after the third insertion B becomes empty (terminating the process), we model the decision space using the first two steps to estimate average branching.

The number of insertion positions available at each step depends on the current length of A:

- First insertion: `length(A) + 1 = 6 + 1 = 7` possible positions.
- Second insertion: on average `E[length(A1)] + 1` positions.
- Third insertion: on average `E[length(A2)] + 1` positions.

Thus, the estimated **total number of decision paths** (i.e., sequences of insertion choices) is approximately:

$$((\text{length}(A) + 1) \cdot (E[\text{length}(A_1)] + 1) \cdot (E[\text{length}(A_2)] + 1)) \cdot P(13, 6) \cdot 13^3$$

Plugging in the values: (7 * 5.86 * 5.16) ≈ 211.5

Compare this to the static no-matching case, which had $\binom{9}{3} = 84$ final arrangements. The ratio is: 211.5/84 ≈ 2.5

This means the **number of decision paths is roughly 2.5 times larger** than the number of "final static combinations" under the no-matching assumption.

> **Crucially**, this estimate does **not** count final card combinations—it counts **decision sequences** (i.e., dynamic strategies). Because the simulation accounts for matching rules and the order-dependent nature of insertions (e.g., inserting B[0] then B[1] can yield a different outcome than the reverse), it captures the true **dynamic combinatorial complexity** of the game.

Therefore, the actual search space for optimal play—represented as a decision tree—is significantly larger than the static upper bound previously calculated. This highlights the immense scale of the problem and underscores the challenge in exhaustively searching for optimal strategies.

Exhaustive recursive search / brute-force recursion (Recursive_StrategyAndScore) attempts all possible options to guarantee finding the optimal solution, resulting in an enormous decision tree. However, the Verification Complexity Analysis yields a bound of $O(n^{3/2})$, indicating that the quality of any given strategy can be verified efficiently.

The problem is in NP and is likely NP-hard.

This resembles the Travelling Salesman Problem (TSP), where the decision to visit a particular city directly affects the distances and path choices for subsequent cities. Similarly, in our problem, a single "matching" operation fundamentally alters the "map" (i.e., list A), thereby influencing the scores (i.e., optimal card-playing decisions) of all subsequent "paths." This structural similarity strongly suggests that our problem is NP-hard.

NP-hardness Proof

Proof: The Card Game Optimization Problem (CARD_GAME_OPT) is NP-hard

In conducting this proof, we follow the standard conventions of theoretical computer science. The input size of a problem instance is defined as the total number of bits required for its **standard binary representation**. Therefore, the input size of a numerical value N is O(log N). The time complexity of algorithms will be measured based on this input size.

1. Discussion on the Variant: "Is There a Strategy to Achieve Exactly K Points?"

We leverage the **Subset Sum problem**, a known NP-hard problem, and employ a standard reduction structure using Karp reduction to prove the NP-hardness of this variant.

Step 1: Define Related Problems

1. **Card Game Optimization Problem (`CARD_GAME_OPT`)**
    - **Instance:** Two initial card lists `A` and `B`, where card values are positive integers.
    - **Problem:** Find a card-playing strategy that maximizes the final total score.
2. **Card Game Decision Problem with Exact Match (`CARD_GAME_DEC_EQ`)** To perform the reduction, we define the "exact equality" decision version:
    - **Instance:** Two initial card lists `A` and `B`, and a target integer `K`.
    - **Problem:** Does there exist a card-playing strategy such that the final total score **exactly equals** `K`?
3. **Subset Sum Problem** This is a classic NP-complete problem [1]. We specifically refer to its form dealing with distinct integer sets to ensure the validity of the reduction.

- **Instance:** A set of `n` distinct positive integers $S = s_1, s_2, \ldots, s_n$, and a target integer `T`.
- **Problem:** Does there exist a subset `S'` ⊆ `S` such that the sum of all elements in `S'` exactly equals `T`?

Step 2: Construct the Reduction

Our goal is to construct an algorithm that can, in polynomial time, transform any instance `(S, T)` of the Subset Sum problem into an equivalent instance `(A, B, K)` of the Card Game Decision Problem.

Given a Subset Sum instance `(S, T)`, we construct an instance of `CARD_GAME_DEC_EQ` as follows:

1. **Choose a sufficiently large base number `X`:**,since all numbers are given in binary, `X` can be represented in O(log(sum(S))) bits, which is polynomial in the input size.

   Let $Sum(S)$ be the sum of all elements in set `S`. We choose an integer `X` such that $X > Sum(S)$. Since `T` must not exceed $Sum(S)$ for the Subset Sum problem to be meaningful, `X` is necessarily greater than both `T` and any element in `S`. This `X` can be computed and represented in polynomial time (e.g., take $Sum(S) + 1$).

2. **Construct initial lists `A` and `B`:**

   - `A = [ X ]`
   - `B = [ X ] + S` (where `+` denotes list concatenation)

3. **Set the target score `K`:**

   - `K = 2X + T`

This transformation process (computing `X`, creating `A` and `B`) involves only basic arithmetic and list operations, with time and space complexity polynomial in the size of input `S`. Therefore, this is a polynomial-time reduction.

Step 3: Prove the Correctness of the Reduction

Now we must prove: **The Subset Sum problem has a solution if and only if the constructed Card Game Decision Problem has a solution.**

(⇒) Sufficiency Proof

If the Subset Sum problem has a solution, then there exists a strategy in the card game that scores exactly `K`.

If the Subset Sum problem has a solution, then there exists a subset $S' \subseteq S$ with element sum $Sum(S') = T$. We can construct the following strategy:

1. Through a series of operations, the player arranges all cards in `S'` between the two `X` cards, forming a scoring structure `[X, S', X]`.
2. According to the game's matching rules (as described in the main text), this structure produces a score of $2X + Sum(S')$.
3. Since $Sum(S') = T$, the score from this step is $2X + T = K$.
4. After this scoring, both `X` cards and all cards in `S'` are removed. The remaining cards on the field are from set $S \setminus S'$. Because the elements in the original set `S` are distinct, these remaining cards have

distinct values and cannot form any matching scores.
5. Therefore, the final total score is **exactly** `K`.

(⇐) Necessity Proof

If there exists a strategy in the card game that scores exactly `K`, then the Subset Sum problem has a solution.

If there exists a strategy that achieves a final score of exactly $K = 2X + T$, let's analyze the source of the score. In this constructed instance, the only cards that can form a match are the two `X` cards.

1. **Case 1: The two `X` cards match directly adjacent.** If the strategy makes the two `X` cards directly adjacent forming an `[X, X]` structure, then the score will be `2X`. Afterward, only the distinct cards from set `S` remain on the field, which cannot score further. The final total score is `2X`. But this does not equal $K = 2X + T$ (since `T` is a positive integer). Therefore, this strategy cannot achieve the target score `K`.

2. **Case 2: The two `X` cards match non-adjacently.** For the two `X` cards to match, there must be some cards from `S` between them. Let the subset of cards sandwiched in between be $S_{mid} \subseteq S$. The scoring structure formed is `[X, S_mid, X]`.

   - The score from this structure is $2X + Sum(S_{mid})$.
   - After scoring, the remaining cards on the field cannot match again. Therefore, this is the final total score.
   - For the total score to exactly equal `K`, we must have: $2X + Sum(S_{mid}) = K = 2X + T$.
   - This directly yields $Sum(S_{mid}) = T$.

   This means we have found a subset of `S` (S_{mid}) whose element sum equals `T`. Therefore, the Subset Sum problem has a solution.

We have proven the equivalence in both directions. Since `Subset Sum` is NP-complete, and we have constructed a polynomial-time reduction to `CARD_GAME_DEC_EQ`, we conclude:

Conclusion i: The decision variant of the card game problem (`CARD_GAME_DEC_EQ`) is NP-hard.

> Although this paper focuses on the optimization problem CARD_GAME_OPT, it is worth mentioning that since its variant decision problem is NP-hard, and because any given card-playing strategy can be simulated and its score verified in polynomial time ($O(n^{3/2})$), it belongs to NP. Therefore, `CARD_GAME_DEC_EQ` is NP-complete.

Relationship Between Optimization Problem, Decision Problem, and Their Variants

Now we explore the intrinsic connections between three problems:

- **Variant Decision Problem**: As proven above, "Does there exist a strategy such that the total score exactly equals a given value K?" (proven to be NP-hard)
- **Decision Problem**: Directly corresponding to the optimization problem, "Does there exist a strategy such that the total score is at least a given value K?" (∃ strategy_score ≥ K)
- **Optimization Problem**: The core problem studied in the project, "Maximize the total game score."

These three have tight computational complexity connections:

1. We proved that the "score = K" variant decision problem is NP-hard. This reveals the inherent computational difficulty of the game rules—even precisely achieving a specific score is theoretically challenging.

2. The "score = K" problem is a special case of the "score ≥ K" problem. Any algorithm that can solve the more general "score ≥ K" problem must correctly handle all its special cases. Therefore, the worst-case time complexity of solving the "score ≥ K" problem must be no less than the complexity of solving the "score = K" problem. Thus, the "score ≥ K" decision problem directly corresponding to the optimization problem is also NP-Hard.

Conclusion ii: The score decision problem of the card game is NP-hard.

Equivalence of Optimization Problem and Its Decision Problem (Turing Equivalence)

The "score maximization" optimization problem and its directly corresponding "∃ strategy_score ≥ K?" decision problem are Turing equivalent in polynomial time. This means that if we have an "oracle" that can solve either problem, we can solve the other in polynomial time.

- **(Optimization ⇒ Decision):** If we can solve the optimization problem, we obtain the maximum score max_score.

  Compare max_score with K:

  - If max_score ≥ K, then since max_score is a strategy_score, there exists a strategy with strategy_score ≥ K, so the decision result is true.

  - If max_score < K, then ∀strategy_score ≤ max_score < K, yielding ∀strategy_score < K, so the decision result is false.

  Thus we can answer (∃ strategy_score ≥ K?) in constant time.

- **(Decision ⇒ Optimization):** If we can solve the "score ≥ K?" decision problem, we can perform binary search over all possible score ranges with time complexity O(log n), and after logarithmic oracle calls, precisely find max_score.

Conclusion iii: From a computational difficulty perspective, these two problems are equivalent. Proving the NP-hardness of the "score ≥ K" decision problem is equivalent to proving the NP-hardness of the original optimization problem.

The core strategy of this card game, in its various forms, touches the theoretical upper limit of NP problems. Therefore, under the assumption that P≠NP, there exists no efficient (polynomial-time) algorithm to solve this problem.

Thus our logical chain is: Variant decision problem is NP-hard → The decision problem corresponding to the optimization problem is NP-hard → The optimization problem is NP-hard

The generalized version of the card game score maximization problem is NP-Hard.

Discussion on Algorithm Optimization / Proposal of Surrogate Modeling

Caching:
The NP-hard nature of the problem leads to an extremely large decision space. We have already

demonstrated that even for a small-scale instance with `length(A) = 6` and `length(B) = 3`, the decision space remains enormous. Consequently, caching techniques are difficult to apply effectively in implementation, as cache hits are exceedingly rare—a fact already confirmed through experimental validation.

Dynamic Programming (DP):
The state of list A changes after every insertion, and the content of A itself is highly complex. This makes it challenging to identify overlapping subproblems, which are essential for dynamic programming. Moreover, the dynamic and intricate nature of A's state renders it unsuitable as a key for DP state storage and retrieval. Therefore, dynamic programming is not applicable to this specific problem.

Pruning / Heuristic Methods:
Based on our understanding of the matching rules employed in `simulate_insertion_tool`, heuristic methods can indeed be leveraged for optimization at certain scales to prune recursive branches. To this end, I introduced a feature called `future_score`, which enables such pruning. The specific implementation details are elaborated in the genetic algorithm section.

Surrogate Modeling:
During this process, the `Recursive_StrategyAndScore` algorithm generates output data, which is then used as a training set to train an AI model. Once trained, the AI model learns the behavioral patterns of the original algorithm and can, within an acceptable margin of error, predict the algorithm's output for new inputs—without actually executing the original (computationally expensive) algorithm. This surrogate modeling approach is the central focus of this discussion.

Part II: AI Technical Details

Software and Hardware Specifications

Hardware:

- CPU: AMD Ryzen 9 5900HS (8 Cores, 16 Threads)
- GPU: NVIDIA GeForce RTX 3050 Laptop (4 GB VRAM)
- RAM: 16 GB

Software:

- Operating System (OS): Windows 11
- IDE: PyCharm
- Version Control: Git
- Environment Management: conda 25.1.1
- Python Version: 3.12.9

> **Note:** For the problem scale addressed in this work, the 4 GB of GPU memory is not a bottleneck. Although it did influence my choices regarding model hyperparameter configuration, the available hardware is sufficient for the models discussed in this paper.

Model Selection

I selected the **Transformer** [3] and **Genetic Algorithm (GA)** [4] as AI models to explore this problem.

The reasons for choosing the **Genetic Algorithm** are twofold:
First, the decision sequence space is extremely large, and each decision depends on the outcome of the previous step. Genetic algorithms excel at searching for optimal solutions in complex, high-dimensional, discontinuous, or non-differentiable search spaces—precisely the characteristics of our problem.
Second, GAs are relatively straightforward to implement and computationally inexpensive, enabling rapid prototyping and quick results.

The **Transformer** was chosen for similar reasons related to interdependent decisions. In this problem, the three-card playing strategy involves sequential actions where each play dynamically modifies list A, thereby influencing subsequent decisions. The Transformer's self-attention mechanism can effectively capture long-range dependencies and interactions between any two elements in the sequence—for example, how the first insertion affects the second and third strategic choices. This capability is crucial for deriving a high-scoring playing strategy.

Additionally, there is a practical hardware consideration: Transformer inference and training are GPU-intensive, whereas Genetic Algorithms are primarily CPU-intensive. By integrating both approaches, we can achieve hardware-level parallelism—leveraging both the CPU and GPU simultaneously during offline computation to maximize hardware utilization.

I used a recursive exhaustive search algorithm to generate a training dataset, which was then used to train both the Genetic Algorithm and the Transformer models. The final surrogate model was constructed by integrating the outputs of these two models.

1. Training Data Acquisition and Preprocessing

I experimented with datasets of various sizes and ultimately decided to use approximately **10,000 samples** to train the Genetic Algorithm (GA) and about **5 million samples** to train the Transformer model.

For the Genetic Algorithm, the entire dataset does not directly participate in "training" in the conventional sense. Instead, the GA stochastically samples from the dataset during its evolutionary process for evaluation and selection. Therefore, the key requirement for the GA dataset is **sufficient coverage and uniformity** across the problem space—rather than sheer volume.

In contrast, for the Transformer, larger datasets generally lead to better model performance. However, due to constraints on training time and hardware resources, extensive testing showed that **5 million samples** strike a good balance: it is large enough to yield strong performance, while further increasing the dataset size becomes impractical and yields diminishing returns.

It is important to emphasize a key distinction: **the Genetic Algorithm and the Transformer utilize the training set in fundamentally different ways**.

The Genetic Algorithm (GA) works by iterating over every possible insertion position and using a genome (i.e., a set of heuristic parameters) to evaluate the "value" or fitness of inserting an element from **B** at that position. The training set provides a fixed collection of **(A, B)** pairs, which serves as a benchmark environment for evaluating and selecting effective genomes.

Importantly, the GA is fundamentally a **heuristic search method**—it still explores the space of possible insertion strategies (though not exhaustively), and for each candidate move (e.g., inserting `B[i]` into a specific position in `A`), it applies its own internal scoring logic to estimate the move's quality. It then uses a **greedy selection** strategy to pick the move with the highest estimated value at each step.

Notably, **the GA does not use the `max_score` or `best_moves` fields from the training set** during its operation. Instead, it relies solely on its evolved heuristic to guide decisions.

In contrast, the **Transformer model** directly leverages the full training data: it learns from the triplets **(A, B, best_moves)**. By observing a large number of such examples, the Transformer attempts to learn the underlying mapping

$$(A, B) \longmapsto \text{best_moves}.$$

It does **not perform any explicit enumeration or search**. Instead, it approximates the optimal strategy implicitly through pattern recognition. It is possible that the learned mapping effectively replicates the behavior of the original exhaustive recursive algorithm—or perhaps discovers an equally effective but more efficient implicit strategy. However, due to the complexity and opacity of deep neural networks, it is difficult to determine exactly what computation the model has internalized.

We further analyze this ambiguity—particularly the tension between the Transformer's architectural inductive biases and the algorithmic nature of the target function—in the section titled **"Conflict in Transformer Modeling Paradigms and Analysis of Inductive Biases."**

Dataset generation function:

```
generate_training_data(num_samples=10000)
```

(implementation available at source code)

Here is a key point:

The length of `best_moves` is not always 3. This occurs when, after playing two cards from hand B, the remaining card cannot form any scoring combination with hand A—i.e., no further points can be earned regardless of how the remaining card is played. In such cases, `recursive_StrategyAndScore` does not include that move in the strategy. Upon inspecting the generated `best_moves`, we found that approximately 10% of the samples exhibit this behavior (i.e., have fewer than 3 moves).

Thus, we face a critical design decision: for these `best_moves` sequences whose length is not strictly 3, should we skip them entirely or pad them to length 3?
Skipping them might discard valuable underlying relationships in the data, while padding could potentially introduce noise.

This trade-off must be evaluated based on the performance of models trained under each approach. Subsequent experimental validation shows that models trained on padded data consistently outperform those trained on skipped data. A detailed comparative analysis is provided in Section 2 under the subsection titled ***"Dataset Preprocessing: Padding vs. Skipping."***

Therefore, we now assume by default that the dataset has undergone padding processing; specific analytical details will be discussed thoroughly in the AI model evaluation.

Since A and B are randomly generated using the `deal_cards_tool`, the dataset needs to be deduplicated, which can be accomplished using Python's `set`. Subsequently, a detailed inspection of the data format, along with exception handling, is required (source code for details).

Finally, we obtain the dataset for the genetic algorithm, named `data_GA.json`, containing 10,000 samples. Here is an example sample:

```
{
    "A": [10, 3, 4, 2, 12, 9],
    "B": [3, 8, 4],
    "max_score": 55,
    "best_moves": [[0, 0], [1, 1], [2, 5]]
}
```

The dataset for the Transformer, `data_TR.json`, is generated and processed in the same way (see the code in `Get_trainingset_huge.py`source code). However, due to its large size, multiprocessing is employed to accelerate data generation, and the resulting JSON files are subsequently converted to JSONL format to support streaming loading. The number of samples was initially set to 5 million before deduplication.

2.1 Implementation of the Genetic Algorithm

The code implementation can be found at: source code

Overview of the Genetic Algorithm Framework

Genome Composition

The genome is a 6-dimensional weight vector corresponding to six game features:

- Current score
- Removal length
- New length
- Future score
- Sum of elements in the new A
- Number of common elements between A and B

Population Initialization

- Population size: 500 individuals
- 70% initialized within heuristic ranges, 30% completely random
- Diversity is introduced to prevent premature convergence

Selection and Reproduction

- **Elitism**: Top 10% of individuals are directly carried over to the next generation
- **Tournament selection**: Competitive selection of parents
- **Crossover**: Weighted average of parental weights
- **Mutation**: Gaussian noise added; mutation rate is adaptively adjusted

Evolutionary Strategies

Two evolutionary strategies are employed:

- A traditional genetic algorithm
- An island model algorithm [5]
  The algorithm adaptively selects between these strategies based on historical performance.

Fitness Evaluation

- Each individual's genome defines weights for the six features.
- Multi-round gameplay is simulated using these weights, and the average score is returned as fitness.
- For each possible card insertion position, the evaluation score is computed as the dot product between the feature vector and the weight vector.

Optimization Mechanisms

- **GPU acceleration**: Using PyTorch tensor operations
- **Parallel evaluation**: Multiprocessing to evaluate different individuals concurrently
- **Batch processing**: Minimizing GPU invocation overhead
- **Early stopping**: Termination if no improvement is observed over consecutive generations

Training Configuration

- Population size: 500
- Number of generations: 20
- Evaluation rounds per individual: 200

Output

The six weight parameters (genome) of the best-performing individual, which constitute the AI's decision-making strategy model.

Feature Design

Features are designed based on the game's scoring rules:

```
features = [
    score,                  # Subset score under the current strategy
    removal_length,         # Length of matched sequence to be removed
    new_length,             # New length of A after insertion
    future_score,           # Estimated future score
    sum_new_A,              # Sum of elements in the new A after matching
    B_card_proportion,      # Proportion of the played B card's value relative to
remaining B cards
]
```

The genome in the genetic algorithm is a list of six floating-point numbers, representing the weights assigned to these six features when evaluating game decisions.

Since these feature values can vary significantly in magnitude, feature scaling is necessary. However, because the features are generated dynamically during gameplay, standard preprocessing (e.g., precomputed

normalization) is not feasible. Therefore, after converting all features into tensors, mean and variance are computed online, and features are standardized on-the-fly.

The "future_score" Feature

Among the features, `future_score` is particularly novel.

Since the number of cards in B is small (0 to 3), this characteristic enables the use of a heuristic approach. Specifically, `future_score` represents the **potential score achievable from the remaining cards after performing the current insertion action**.

In the game, each turn requires selecting one card from B and inserting it into A. This action may alter A's state, which in turn affects the optimal scoring strategy for subsequent moves. Thus, `future_score` dynamically evaluates the expected payoff of future game states—much like how professional chess players consider how a current move influences their strategic options in later turns.

This makes `future_score` a crucial feature for lookahead planning.

The other features are derived from the return values of `simulate_insertion_tool` (source code). Although `match_found` and `removal_length` contain overlapping information, `match_found` was omitted as a feature to keep the model complexity manageable.

The heuristic function for computing `future_score` follows this logic:

When inserting a card from B into A, after this insertion is performed, the function `calculate_future_score(A, remaining_B)` estimates the future potential score as follows:

1. If A and B have **no common elements**, then `future_score = 0`.
2. If there are **no remaining cards in B**, then `future_score = 0`.
3. If **only one card remains in B**, check whether this card shares any elements with A:
    - If yes, call `get_best_insertion_score` (source code) to compute the best possible score from inserting that card, and use this as `future_score`.
    - If not, `future_score = 0`.

This defines the future score estimation function: **`calculate_future_score(A, remaining_B)`**.

Core Function Documentation

Strategy Generation Function

Source Code

```
def GA_Strategy(genome, A, B):
    """
    Generate a card playing strategy based on genome and card sets A and B.
    For each card in B, simulate all possible insertion positions in A,
    calculate 6 feature tensors, weight features using genome,
    and obtain evaluation value for each position.
```

```
    Sort by evaluation value in descending order to generate final strategy.

    Args:
        genome (list[float]): Weight genes of length 6, used for feature
weighting.
        A (list): Current hand cards or played card sequence.
        B (list): List of candidate cards to be played.

    Returns:
        list: Card playing strategy sorted by evaluation value.
    """
```

Genome Evaluation Function

```
def evaluate_genome(genome, num_rounds, seed_base):
    """
    Calls GA_Strategy function to complete the conversion from A, B and genome
    to corresponding specific strategy. We need to use the same batch of A, B
    for genome evaluation, so a fixed seed_base is required.

    Uses calculate_score_by_strategy function to calculate actual score of
strategy,
    evaluates multiple times, and takes average score as final evaluation result.

    Args:
        genome (list[float]): Genome to be evaluated.
        num_rounds (int): Number of evaluation rounds.
        seed_base (int): Seed base for generating A and B.

    Returns:
        float: Average evaluation score of the genome.
    """
```

Island Model Evolution Function

```
def island_model_evolution(population, fitnesses, pop_size, tournament_size,
                           mutation_strength, num_rounds, islands, migration_rate,
                           generation=0, max_generations=60):
    """
    1. Evenly distribute entire population across 3 islands
    2. Each island evolves independently, calls evaluate_genome to calculate
fitness,
       preserves top 10% individuals with highest fitness in current island,
       directly entering next generation
    3. Use tournament selection to select winner individuals, randomly select
parents
       among them, perform crossover and mutation to generate offspring,
       randomly select from these excellent offspring to replace individuals
       with low fitness in another island
```

```
    Args:
        population: Current population (list)
        fitnesses: Fitness values of current population (list)
        pop_size: Total population size (int)
        tournament_size: Tournament selection size (int)
        mutation_strength: Mutation strength (float)
        islands: Number of islands (int)
        migration_rate: Migration rate (float)

    Returns:
        new_population: Population of all islands after evolution (list)
    """
```

Training Parameters

Main Genetic Algorithm Function

```
def genetic_algorithm(pop_size, generations, num_rounds, elitism_ratio,
                      tournament_size, evolution_methods,
                      early_stop_generations, early_stop_threshold):
    """
    Main genetic algorithm function.

    Args:
        pop_size: Population size
        generations: Maximum number of generations
        num_rounds: Number of rounds to evaluate each genome
        elitism_ratio: Elite proportion
        tournament_size: Tournament selection size
        evolution_methods: List of available evolution methods
        method_probs: Usage probability of each evolution method
        early_stop_generations: Number of consecutive generations for early
stopping
        early_stop_threshold: Improvement threshold for early stopping

    Returns:
        best_genome: Best genome
    """
```

Tested Parameters

After testing, these parameters have yielded sufficiently good results:

```
pop_size = 500
generations = 25
num_rounds = 500
elitism_ratio = 0.1
tournament_size = 3
```

```
evolution_methods = ['island', 'standard']
early_stop_generations = 3
early_stop_threshold = 0.01
```

Genome Analysis

Next, let's analyze the obtained best genome. The following is based on one of my test datasets.

This is a genome obtained after training. The model terminated at generation 16 due to early stopping:

```
[2.5638819522632654, 0.30592210650063667, -1.1848400755378474,
 4.178412919525965, -1.4191214580170772, -0.09006199513519475]
```

Performance Analysis of Evolution Methods

```
island method: average fitness = 49.33, max fitness = 49.90,
               improvement rate = 0.14%, usage count = 6
standard method: average fitness = 49.50, max fitness = 50.30,
                 improvement rate = 4.27%, usage count = 9
```

This indicates that both standard genetic algorithm and island model evolution are effective.

I conducted multiple tests, and it's not certain which method has higher improvement rate, indicating that for different genomes and different test sets, different models are suitable. Moreover, the island model does not show clear superiority in this problem.

The final improvement rates are all relatively low, indicating that the first generation genome already performed well (this is also related to using warm start).

Feature Weight Analysis

```
Genome Feature Weights:
Current score: 2.5639
Removed length: 0.3059
New length: -1.1848
Future score: 4.1784
Sum of new A elements: -1.4191
Value ratio of played card to remaining cards in B: -0.0901
```

Returning to the genome itself, analyzing the weights of its final features:

Feature 1: Current Score (2.5639)

This weight is very high and positive, indicating that the algorithm highly values the score that can be directly obtained from the current operation.

Feature 2: Removed Length After Matching at This Position (0.3059)

This weight is a low positive value, indicating that the algorithm tends to remove fewer elements. This suggests the algorithm tends to avoid removing too many elements. This may be because retaining more elements can provide more possibilities for future moves.

Feature 3: New Length (-1.1848)

Negative value, indicating the algorithm tends to obtain a shorter new A after insert-match operation. This seems to align with the tendency of Feature 2's weight.

> Although the values of Feature 2 and Feature 3 can be obtained through simple mathematical calculation (length of new A = length of old A - length removed due to matching), I cannot guarantee that the AI algorithm can detect this relationship. Moreover, these two states represent different perspectives, so I prefer to retain them. This feature may bring performance improvement or accelerate model convergence.

Feature 4: Future Score (4.1784)

This weight is very high and positive, indicating that the algorithm highly values the impact of moves on possible future scores, demonstrating strong foresight.

Feature 5: Sum of Elements in New A (-1.4191)

The weight is a relatively high negative value, possibly indicating that the algorithm not only considers current score, but also that retaining fewer scoring possibilities may allow subsequent operations to gain advantages.

Feature 6: Value Ratio of Moved Card to Remaining Cards in B (-0.0901)

This feature indicates whether the algorithm tends to play high-value cards first or low-value cards first. The data this time shows a weight close to 0, with very small impact. However, in some tests, this value is less than -1, indicating this feature may achieve some balance with other features.

Then we need to evaluate our genetic algorithm and obtained genome under given real A and B. However, this part will be placed later and evaluated together with the transformer model.

2.2 Implementation of the Transformer Model Algorithm

I employed a standard Transformer model. During training, the inputs are A, B, and $best_moves$. During inference, A and B serve as input features, while $best_move$ acts as the target label for computing the loss function, thereby guiding model optimization (i.e., the supervised learning target).

Initially, I attempted a more streamlined implementation of this Transformer training, but encountered issues such as slow convergence, gradient explosion, and the inability to flexibly terminate training manually. Consequently, in addition to early stopping and gradient clipping, I incorporated learning rate warm-up, cosine annealing, and graceful shutdown with signal handling to accelerate training without compromising performance.

> *: Graceful shutdown proved especially helpful during the initial exploration phase for roughly estimating the training parameter ranges. For instance, once the loss became sufficiently small, I could manually*

> *stop training—continuing would not only waste time but also risk gradient explosion. At that point, obtaining a reasonably good temporary model allowed further analysis to infer better parameter ranges. Although once the optimal training parameters were largely determined, manual interruption became unnecessary and graceful shutdown lost its utility.*

Explanation of Key Functions

[source code] (https://github.com/tcita/python_server/blob/master/AI_algorithm/Trans.py)

Data Preparation and Feature Engineering

```
prepare_data_transformer(sample, num_a=6, num_b=3)
```

- **Purpose**: Handles data preprocessing and feature engineering.
- **Feature Engineering**: For each element in sequences A and B, six features are created (identical to those used in the genetic algorithm).
- After feature augmentation, the original A, B sequences, and corresponding $best_moves$ are packed together into the format required for model training and returned.
- **Parameters**:
  - `sample` (dict): A single data sample containing sequences A, B, and $best_moves$.
  - `num_a` (int): Length of sequence A.
  - `num_b` (int): Length of sequence B.
- **Returns**: `input_sequence` (shape $(9, 6)$) along with corresponding `order_target` and `pos_target` labels.

```
class TransformerMovePredictor(nn.Module)
```

- A trained model is an instance of the `TransformerMovePredictor` class. Training updates the weights and biases of all neural network layers. The model uses two prediction heads to output:

  1. The predicted insertion order of elements in B,
  2. The predicted insertion positions in A for each element of B.

- **Input (`src`)**:
  - A tensor of shape $(batch_size, 9, input_dim)$.
  - Sequence length is fixed at 9: the first 6 elements represent contextual information from A, and the last 3 represent target elements from B.
  - $input_dim$ denotes the feature dimensionality of each element.
- **Output**:
  - A tuple `(order_logits, pos_preds)`:
    - `order_logits`: Tensor of shape $(batch_size, 3, 3)$, used to predict the relative order among elements in B.

    - pos_preds: Tensor of shape $(batch_size, 3)$, used to predict the insertion position in A for each element of B.

```
Transformer_predict(A, B, model, num_a=6, num_b=3)
```

- Performs prediction using a trained Transformer model (specifically for $|A| = 6$, $|B| = 3$).
- Employs a heuristic check: if A and B are disjoint and B contains no duplicate elements, scoring is impossible. In such cases, prediction is skipped and a default move strategy is returned directly. (Note: during training, samples where A and B are disjoint were already excluded.)
- Intuition: If $A \cap B = \emptyset$ and B has no duplicates, no matches can be formed. However, if B contains duplicates (e.g., $b_1 = b_2$), each copy can be inserted into different positions in A, potentially achieving a maximum score of $\mathbf{sum}(A) + \mathbf{sum}(B)$.
- **Parameters**:
    - A (numpy.ndarray): Array representing sequence A.
    - B (numpy.ndarray): Array representing sequence B.
    - model (nn.Module): Trained Transformer model.
    - num_a (int): Length of sequence A.
    - num_b (int): Length of sequence B.
- **Returns**: (List of best move strategies, predicted score)

Positional Encoding in Transformer

```
class PositionalEncoding(nn.Module)
```

- Adds positional encoding vectors (retrieved from a precomputed buffer) to the input token embeddings, then applies dropout. This injects crucial sequential order information into the model.
- The positional encodings are **not learned parameters**; instead, they are generated using fixed sine and cosine functions.
- **Args**:
    - d_model (int): Dimensionality of the model (i.e., embedding dimension).
    - dropout (float): Dropout probability (default: 0.1).
    - max_len (int): Maximum supported sequence length (default: 5000).
- **Output**: $(batch_size, seq_len, d_model)$

I have reflected on the rationale behind choosing sine and cosine functions for positional encoding.

In the standard positional encoding, the angular term is defined as:

$$\theta_{\text{pos},i} = \text{pos} \cdot 10000^{-2i/d_{\text{model}}}$$

Thus, $\omega_i = 10000^{-2i/d_{\text{model}}}$, and $\theta_{\text{pos},i} = \text{pos} \cdot \omega_i$.

1. **Multi-scale Frequency Characteristics**

The angular term forms a 2D matrix, and this design inherently exhibits **multi-scale frequency characteristics**:

- **Along the position dimension (rows)**:
  For a fixed frequency index i, $\theta_{\text{pos},i}$ grows linearly with the position index pos.
  - When i is small (corresponding to lower dimensions of the encoding vector), the decay factor $10000^{-2i/d_{\text{model}}} \approx 1$, resulting in a large slope. Consequently, $\sin(\theta)$ and $\cos(\theta)$ oscillate rapidly, enabling fine-grained discrimination between adjacent positions.
  - When i is large (corresponding to higher dimensions), the decay factor becomes significantly smaller than 1, causing θ to increase slowly. The sine and cosine functions thus vary smoothly, encoding coarse-grained, long-range global positional information.
- **Along the embedding dimension (columns)**:
  For a fixed position pos, $\theta_{\text{pos},i}$ **decays exponentially** as the dimension index i increases, because

  $$10000^{-2i/d_{\text{model}}} = \exp\left(-\frac{2i}{d_{\text{model}}} \ln 10000\right)$$

  is an exponential decay function with respect to i.

This design enables the model to simultaneously capture both local details and global structural information, enhancing its capacity to represent features at multiple scales and thereby improving its ability to model complex patterns.

2. **Linear Representability of Relative Positions**

For any fixed offset k, the positional encoding at position $\text{pos} + k$, denoted $PE(\text{pos} + k)$, can be expressed as a **linear transformation** of $PE(\text{pos})$—specifically, a rotation matrix.

The sine-cosine positional encoding assigns to each pair of dimensions $(2i, 2i+1)$ the vector:

$$\mathbf{p}_{\text{pos}}^{(i)} = [\sin(\text{pos} \cdot \omega_i) \;\; \cos(\text{pos} \cdot \omega_i)].$$

For a relative offset k, we have:

$$

$$[\sin((\text{pos} + k)\omega_i) \;\; \cos((\text{pos} + k)\omega_i)]$$

\underbrace{

$$[\cos(k\omega_i) \quad \sin(k\omega_i) \; -\sin(k\omega_i) \quad \cos(k\omega_i)]$$

}_{\mathbf{R}(k\omega_i)^\top}

$$[\sin(\text{pos}\omega_i)\ \cos(\text{pos}\omega_i)]$$

, $$

where

$$\mathbf{R}(\theta) = [\cos\theta \quad -\sin\theta\ \sin\theta \quad \cos\theta]$$

is the standard 2D rotation matrix.

This means the encoding at position $\mathbf{pos} + \boldsymbol{k}$ can be obtained by left-multiplying the encoding at position $\mathbf{pos}$ with $\mathbf{R}(\boldsymbol{k}\omega_i)^\top$.

Because the self-attention mechanism fundamentally computes dot products, and dot products are well-behaved under linear transformations, this property allows the model to **easily learn relative positional relationships**.

Therefore, the alternating use of sine and cosine functions enables the self-attention mechanism to effortlessly capture and reason about relative positional information—such as “shift $\boldsymbol{k}$ positions forward” or “shift $\boldsymbol{k}$ positions backward.”